Selection Sort: One Exchange for All

By Matthew Lyons

Original presentation from September 2024

slidesmania.com

I plan to explain...

01 THE BASICS

Selection Sort
and its intricacies

02 MY CODE

Sorting, printing,
etc.

03 MY RESULTS

The code's outputs
and my takeaways

slidesmania.com

01

First...

What Is Selection Sort?

(Time complexity: n^2)

In terms of pseudocode, Selection Sort takes an array *S[n]* and the size of the array as an integer *n* as inputs. Using these parameters, it keeps track of the index of the smallest element of the array and recursively swaps it with the element at *S[i]*.

slidesmania.com

02

Second...

What Code Did I Write?

The next two slides will showcase my Selection Sort function code.

slidesmania.com

```
// Selection Sort function:
void selectionSort(int arr[], int n, long int comp_count, long int swap_count, long int total_count) {
    // Initialize indices to be used for comparing and swapping
    int smallest, temp;

    /* First "for" loop: "i" starts at 0 and iterates through the array
    until it is one less than the length of the array.
    It will give its value to "smallest." */
    for (int i = 0; i < n - 1; i++) {
        smallest = i;
        /* Second "for" loop: "j" starts at 1,
        iterating until it is the length of the array. */
        for (int j = i + 1; j < n; j++) {
            comp_count++;
            total_count++;
            if (arr[j] < arr[smallest]) {
                smallest = j;
            }
        }
```

smanja.com

```
        if (smallest != i) {
            /* The value at "i" is swapped with the value at "smallest."
               "smallest" will be "i" unless the element at index "j"
               is smaller than the current value of "smallest." */
            temp = arr[i];
            arr[i] = arr[smallest];
            arr[smallest] = temp;
            swap_count++;
            total_count++;
        }
    }

    cout << "Total Comparisons = " << comp_count << endl << "Total Swaps = " << swap_count << endl << "Total Comparisons and Swaps = " << total_count << endl;
}
```

slidesmania.com

```
Almost-sorted array:
{1, 9, 6, 19, 23, 52, 42, 38, 61, 86}
Total Comparisons = 45
Total Swaps = 2
Total Comparisons and Swaps = 47
Sorted array:
{1, 6, 9, 19, 23, 38, 42, 52, 61, 86}
Time taken = 7 microseconds


Unsorted array:
{10, 32, 3, 89, 85, 50, 62, 11, 9, 82}
Total Comparisons = 45
Total Swaps = 5
Total Comparisons and Swaps = 50
Sorted array:
{3, 9, 10, 11, 32, 50, 62, 82, 85, 89}
Time taken = 9 microseconds
```

(10-element)

Almost-Sorted and Unsorted Arrays

The algorithm handles both the Almost-Sorted Array and the Unsorted Array well.

slidesmania.com

```
Reverse-Sorted array:
{95, 78, 75, 74, 59, 44, 29, 25, 14, 7}
Total Comparisons = 45
Total Swaps = 5
Total Comparisons and Swaps = 50
Sorted array:
{7, 14, 25, 29, 44, 59, 74, 75, 78, 95}
Time taken = 13 microseconds


Duplicate-containing array:
{5, 76, 94, 5, 94, 62, 66, 76, 11, 90}
Total Comparisons = 45
Total Swaps = 6
Total Comparisons and Swaps = 51
Sorted array:
{5, 5, 11, 62, 66, 76, 76, 90, 94, 94}
Time taken = 6 microseconds
```

(10-element)

Reverse Array and Array with Duplicates

The algorithm handles both the Reverse-Sorted Array and the Array with Duplicates well.

slidesmania.com

03

Third...

My Takeaway From My Code

While I was in the testing phase of writing my code, I noticed that I had problems with properly tracking algorithm swaps and run time. I debugged these issues with the help of Dr. Williams and AI influence.

slidesmania.com